

Name: _____

Exam #2

Note:

All the codes are supposed to run on a 32-bit Cortex-M4 processor in Thumb mode if not specified.

You may need the ASCII table, which is on the last page.

Condition Codes

Please fill in an immediate value to complete the following instructions to set/clear the conditional flags as stated. There might be multiple valid values, you only need to write down ONE possible solution.

1. (3 PTS) Fill an immediate value in the blank to let N = 1, Z = 0, C=0, and V = 1.

```
MOV32    R1, #0x7E000000
MOV32    R2, _____
ADDS     R0, R1, R2
```

2. (3 PTS) Fill an immediate value in the blank to let N = 0, Z = 0, C=1, and V = 0.

```
MOV32    R1, #0x22000000
MOV32    R2, _____
ADDS     R0, R1, R2
```

3. (3 PTS) Fill an immediate value in the blank to let N = 1, Z = 0, C=1, and V = 0.

```
MOV32    R1, #0xDD000000
MOV32    R2, _____
ADDS     R0, R1, R2
```

4. (3 PTS) Fill an immediate value in the blank to let N = 1, Z = 0, C=0, and V = 0.

```
MOV32    R1, #0x10000000
MOV32    R2, _____
SUBS     R0, R1, R2
```

Shift and Rotate Operations Codes

5. (4 PTS) Predict the results of the following shift operations (hexadecimal value in R0), and the condition flags. Assume all the flags are cleared before the operations. Please show your work.

```
MOV32    R1, #0x98760000
MOV32    R2, #0x4
ASRS     R0, R1, R2
```

ANS: _____
N: _____; Z: _____; C: _____.

6. (4 PTS) Predict the results of the following instructions (hexadecimal value in R0). Please show your work.

```
MOV     R1, #137
MOV     R2, #128
SUB     R0, R1, R2, LSR #4
```

ANS: _____

Logical Operations

7. (4 PTS) What is the hexadecimal result of the following **ORR** (OR) operation? (Show your work.)

#0x36 | #0x81 = _____

8. (4 PTS) What is the hexadecimal result of the following **EOR** (exclusive OR) operation? (Show your work.)

#0x27 ^ #0xA2 = _____

9. (7 PTS) Assume there is an ASCII value of a letter in register R0, and you want to check if it is in the range of "A" to "O", or in the range of "P" to "Z". Please design a **mask** (an

immediate value) to extract the required information, and complete the following code.

```
MOV    R1, _____  
AND    R3, R0, R1  
CMP    R3, _____  
BEQ    range_AO  
CMP    R3, _____  
BEQ    range_PZ  
BNE    end_loop
```

Directives

10. (3 PTS) Write the **directive** to create a **16-bit constant** named Const1 initialized to **0x8000**.

11. (3 PTS) Write the **directive** to create a **16-bit variable** named **Var1**.

12. (6 PTS) Please fill in the values in the according memory after the following directives. Assume the address of **Const2** is 0x00000284. If the memory is not defined or initialized, fill in with "00". (Using hexadecimals for all the values, don't need to write the prefix of "0x"; one cell means a byte.)

```
Const2    DCD    0x56781234, 0xABCD2468  
Const3    DCW    0x1357, 0x2024
```

Address	Contents							
0x00000284								
0x0000028C								

RAM/ROM Access

13. (4 PTS) Write the assembly code to **load** register **R0** with the value of the **16-bit** constant named **const1**.

14. (6 PTS) Assume we have defined the **Const2** and **Const3** and initialized as in the question #12. Please predict the values in register R0 after running the following codes.

```
LDR    R1, =Const2
LDRH   R0, [R1, #4]
```

ANS (R0): _____

Conditional Branching and Execution

15. (4 PTS) Write the single instruction to SUB (subtract) R2 from R3 and place the result in R1 only if the Negative flag is set (**N = 1**). Update the flags in the instruction.
16. (4 PTS) Write the single instruction to ADD R2 to R3 and place the result in R1 only if the negative flag and the overflow flag are different (**N != V**).

Programming

17. (7 PTS) Give the final value (in hexadecimal) of register R0 and R1 after the following code is executed. Assume that all condition flags are clear (=0) before beginning execution. Please show how you get the results.

R0 = _____
R1 = _____

```
; Define constant
Input      DCD    5
...
; Code
          MOV     R0, #1
          LDR     R2, =Input
          LDR     R1, [R2]
```

```
lbegin
        MUL    R0, R1
        SUBS   R1, #1
        BHI    lbegin
Lend
        NOP
```

18. (8 PTS) Convert the following C code snippets to ARM assembly. Assume all variables are 32 bits. You do not have to define the variables, but you must load/store their values from/to RAM. You are NOT allowed to use any Branch instructions for this problem (use conditional execution).

```
if(varX >= varY) {
    varZ = 0;
else
    varZ = 1;
}
```

19. (10 PTS) Please count how many letter “t” in the following quote. Assume the quote is defined in the memory as

```
myString    DCB    "O Captain! my Captain! our fearful trip is done",0
```

You must use a loop, and determine the end of the myString by checking control symbol “NUL”, which is the number 0 in the instruction. You need to place the results into a **variable** named Num1.

20. (10 PTS) Bubble sort, or sinking sort, is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. (You can refer to the following C code.) Write the assembly code to assort the numbers in the array named **TESTDATA** in ascending order and place the result in the same space.

For example, if the numbers in **TESTDATA** are 2, 9, 3, 8, 7 and stored in the memory space from 0x20000000 to 0x20000013 (4-bytes/32-bits each number), the data in the same memory space after running the code should be 2, 3, 7, 8, 9.

Assume the **TESTDATA** has been defined, and the size of the array (how many numbers in the array) is saved in the variable named **ArraySize**.

```
for(i = 0; i < n; ++i)
{
    for(j = 0; j < n+1; ++j)
    {
        if(a[i] > a[i+1])
        {
            temp = a[i];
            a[i] = a[i+1];
            a[i+1] = temp;
        }
    }
}
```

ASCII (1977/1986)																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
Changed or added in 1963 version Changed in both 1963 version and 1965 draft																